Mạng encoder–decoder trong xử lý ngôn ngữ tự nhiên

Từ chuỗi token đầu vào đến chuỗi token đầu ra

Báo cáo lý thuyết gắn với tài liệu và notebook của dự án

08/10/2026

Tóm tắt

Báo cáo giải thích mạng encoder–decoder cho bài toán biến đổi một chuỗi văn bản thành chuỗi khác. Tài liệu trình bày vai trò của token, embedding, trạng thái ẩn, phép truyền xuôi, dịch phải chuỗi đích, teacher forcing, hàm mất mát, lan truyền ngược và sinh văn bản khi suy luận. Các phương trình dùng RNN đơn giản để làm rõ cơ chế; attention và Transformer được giới thiệu như bước mở rộng. Ví dụ dịch trong báo cáo chỉ minh họa luồng dữ liệu, vì dự án chưa huấn luyện mô hình dịch máy.

Mục lục

1 Mục tiêu của mạng encoder–decoder **1**

2 Chuẩn bị token và embedding **2**

3 Encoder và decoder là mạng nơ-ron ở đâu? **2**

3.1 Encoder RNN 2

3.2 Decoder RNN và tầng dự đoán từ 2

4 Một cặp câu đi qua mô hình như thế nào? **3**

5 Loss, lan truyền ngược và cập nhật tham số **3**

6 Sinh chuỗi khi suy luận **3**

7 Từ nút thắt một vector đến attention **4**

8 Phạm vi của dự án và cách kiểm tra một triển khai mới **4**

Tài liệu tham khảo **4**

1 Mục tiêu của mạng encoder–decoder

Cho câu nguồn $x = (x_1, \ldots, x_T)$ và câu đích $y = (y_1, \ldots, y_U)$, mô hình học phân phối có điều kiện $P(y \mid x)$. Độ dài T và U có thể khác nhau. Encoder đọc nguồn và tạo biểu diễn H; decoder dựa vào H cùng các token đã sinh để dự đoán token kế tiếp:

$$P(y \mid x) = \prod_{t=1}^{U+1} P(y_t \mid y_{<t}, H), \qquad H = f_\theta(x), \tag{1}$$

với $y_{U+1} = \langle\text{EOS}\rangle$. Các tham số θ và ϕ của encoder và decoder được học cùng nhau từ các cặp câu nguồn–đích. Đây là kiến trúc cho các nhiệm vụ như dịch máy, tóm tắt và diễn đạt lại. Tên gọi "mã hóa" ở đây nghĩa là tạo biểu diễn phục vụ dự đoán, không phải mã hóa bảo mật.

Token nguồn $x_{1:T} \longrightarrow$ Encoder $\longrightarrow$ Biểu diễn $H \longrightarrow$ Decoder $\longrightarrow$ Token đích $y_{1:U}$

2 Chuẩn bị token và embedding

Một câu phải được tách token bằng quy tắc nhất quán, rồi ánh xạ mỗi token sang ID trong từ vựng. Token ID chỉ là chỉ số; độ chênh giữa hai ID không có ý nghĩa ngôn ngữ. Với từ vựng nguồn kích thước V_s và chiều embedding d_s, ma trận $E_s \in \mathbb{R}^{V_s \times d_s}$ biến ID x_t thành vector $E_s[x_t]$. Decoder có thể dùng từ vựng đích và ma trận $E_t \in \mathbb{R}^{V_t \times d_t}$ riêng.

Token đặc biệt	Công dụng	Lưu ý khi huấn luyện/suy luận
$\langle$BOS$\rangle$	Khởi động decoder	Chỉ là đầu vào ở bước đầu
$\langle$EOS$\rangle$	Báo kết thúc chuỗi đích	Là nhãn hợp lệ cần dự đoán
$\langle$PAD$\rangle$	Làm đều chiều dài batch	Loại khỏi loss và attention phù hợp
$\langle$UNK$\rangle$	Đại diện token ngoài từ vựng nếu có	Không thay thế được mọi thông tin của từ mới

Bảng 1: Các token điều khiển một bài toán sinh chuỗi cơ bản.

Từ vựng và phép chuẩn hóa văn bản phải được xác định từ train. Khi dùng subword, một từ mới có thể tách thành các mảnh đã biết; đó là một lựa chọn token hóa khác với từ vựng theo từ nguyên khối. Báo cáo `token_embed.tex` giải thích sâu hơn về embedding và cách học vector từ.

3 Encoder và decoder là mạng nơ-ron ở đâu?

3.1 Encoder RNN

Một RNN đơn giản cập nhật trạng thái khi đọc từng token:

$$h_i = \tanh(E_s[x_i]W_x + h_{i-1}W_h + b_h)\,, \qquad h_0 = 0. \tag{2}$$

Nếu embedding có chiều d_s và hidden state có chiều h, thì $W_x \in \mathbb{R}^{d_s \times h}$, $W_h \in \mathbb{R}^{h \times h}$, $b_h \in \mathbb{R}^h$. Phép nhân với các ma trận trọng số, cộng bias và áp dụng tanh chính là tính toán của các nơ-ron. Cùng bộ tham số được lặp lại ở mọi vị trí nguồn. Encoder cơ bản trả trạng thái cuối $c = h_T$; bản có attention giữ toàn bộ $H = (h_1, \ldots, h_T)$.

3.2 Decoder RNN và tầng dự đoán từ

Một decoder minh họa khởi tạo $s_0 = c$ và tính

$$s_t = \tanh(E_t[\tilde{y}_{t-1}]U_x + s_{t-1}U_h + b_s)\,, \tag{3}$$

$$o_t = s_t W_o + b_o, \tag{4}$$

$$P(y_t = j \mid y_{<t}, x) = \frac{\exp(o_{t,j})}{\sum_{k=1}^{V_t} \exp(o_{t,k})}. \tag{5}$$

Ở đây $\tilde{y}_0 = \langle\text{BOS}\rangle$. Nếu hidden state decoder có chiều h, thì $U_x \in \mathbb{R}^{d_t \times h}$, $U_h \in \mathbb{R}^{h \times h}$, $W_o \in \mathbb{R}^{h \times V_t}$ và $b_o \in \mathbb{R}^{V_t}$. Trường hợp kích thước trạng thái encoder khác decoder cần thêm một phép chiếu để tạo s_0.

Vector o_t là logits; softmax đổi nó thành xác suất trên từ vựng đích. Mạng LSTM/GRU dùng công thức cập nhật trạng thái phức tạp hơn nhưng vẫn giữ vai trò đọc nguồn và sinh đích [1].

4 Một cặp câu đi qua mô hình như thế nào?

Lấy ví dụ minh họa $x =$ "tôi yêu mèo" và $y =$ "i love cats". Encoder đọc từng token nguồn và tạo c. Decoder nhận BOS trước khi đoán "i". Sau đó cách chọn đầu vào decoder phụ thuộc chế độ hoạt động: Đưa trực tiếp token cần đoán ở bước hiện tại vào decoder sẽ làm lộ đáp án. Teacher forcing sử dụng token

Bước	Input decoder khi train	Nhãn cần đoán	Input khi suy luận
1	⟨BOS⟩	i	⟨BOS⟩
2	i thật	love	Token dự đoán bước 1
3	love thật	cats	Token dự đoán bước 2
4	cats thật	⟨EOS⟩	Token dự đoán bước 3

Bảng 2: Dịch phải chuỗi đích: token đúng ở bước trước là đầu vào khi huấn luyện.

đúng của bước trước; khi suy luận không có nhãn thật, mô hình phải dùng đầu ra trước đó của chính nó. Ví dụ trên là cặp câu do người viết đặt ra để giảng giải, *không phải kết quả thực nghiệm của một mạng dịch đã huấn luyện*.

5 Loss, lan truyền ngược và cập nhật tham số

Với N cặp câu, các chuỗi đích được thêm EOS và PAD để tạo batch. Gọi $m_{i,t} = 1$ ở vị trí token thật, gồm EOS, và bằng 0 ở PAD; $M = \sum_{i,t} m_{i,t}$. Cross-entropy theo token là

$$\mathcal{L} = -\frac{1}{M} \sum_{i,t} m_{i,t} \log P_{\theta,\phi}(y^*_{i,t} \mid y^*_{i,<t}, x_i). \tag{6}$$

Ở truyền xuôi, encoder tạo c rồi decoder tạo logits. Ở truyền ngược, sai số từ softmax đi qua tầng W_o, các bước decoder, $s_0 = c$, rồi các bước encoder. Vì vậy, loss đầu ra cập nhật được trọng số của *cả hai khối*. Một optimizer như gradient descent áp dụng $p \leftarrow p - \eta\, \partial\mathcal{L}/\partial p$ cho từng tham số p. Tính gradient qua các bước RNN dùng lan truyền ngược qua thời gian; chuỗi dài có thể gây khó khăn cho gradient và khả năng giữ thông tin.

Quy trình đúng là chia dữ liệu thành train/validation/test, học tham số trên train, chọn epoch hoặc cấu hình theo validation, rồi đánh giá test một lần với cấu hình đã chốt. Nếu câu nguồn/đích trùng hoặc gần trùng giữa các tập, kết quả đánh giá có thể quá lạc quan.

6 Sinh chuỗi khi suy luận

Suy luận bắt đầu bằng BOS. Mỗi bước decoder tạo phân phối $P(y_t \mid y_{<t}, x)$; greedy decoding chọn token xác suất cao nhất, đưa token ấy vào bước sau, và dừng khi gặp EOS hoặc chạm giới hạn độ dài. Greedy tối ưu lựa chọn cục bộ, không bảo đảm chuỗi có xác suất tổng thể lớn nhất. Beam search giữ nhiều tiền tố ứng viên nhưng vẫn là tìm kiếm xấp xỉ. Vì decoder đã quen nhận token đúng khi teacher forcing, một token dự đoán sai sớm có thể kéo theo lỗi ở các bước sau.

Sự khác biệt giữa train và suy luận cũng quyết định cách kiểm tra code: không được dùng nhãn đích thật trong hàm sinh văn bản. Trường hợp mô hình không sinh EOS cần có giới hạn độ dài để tránh vòng lặp vô hạn.

7 Từ nút thắt một vector đến attention

Trạng thái cuối $c = h_T$ phải chứa mọi thông tin nguồn mà decoder cần; câu dài có thể làm nút thắt này trở nên khó học. Attention cho decoder truy cập nhiều trạng thái encoder và tính ngữ cảnh riêng ở từng bước [2]:

$$e_{t,i} = \text{score}(s_{t-1}, h_i), \quad \alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j \in \mathcal{V}_x} \exp(e_{t,j})}, \quad c_t = \sum_{i \in \mathcal{V}_x} \alpha_{t,i} h_i. \tag{7}$$

$\mathcal{V}_x$ chỉ gồm token nguồn hợp lệ; PAD bị mask. Transformer dùng self-attention trong encoder và decoder, cùng cross-attention từ decoder sang biểu diễn encoder; decoder còn có causal mask để không nhìn token đích tương lai [3]. Báo cáo `seqtoseq.tex` đi sâu vào các khối này và cách đánh giá một thí nghiệm seq2seq.

8 Phạm vi của dự án và cách kiểm tra một triển khai mới

Notebook `encoder_decoder_minh_hoa.ipynb` hiện có các mạng NumPy nhỏ cho biểu diễn và dự đoán từ, cùng phần giải thích seq2seq bằng công thức. Dự án *chưa có mạng RNN encoder–decoder được huấn luyện để dịch câu*. Vì vậy báo cáo này không đưa ra BLEU, độ chính xác dịch hoặc ví dụ đầu ra giả làm kết quả chạy.

Nếu triển khai mô hình sinh chuỗi, cần kiểm tra shape của embedding/trạng thái/logits, kiểm tra nhãn đích được dịch phải đúng một vị trí, xác nhận PAD không đi vào loss và attention, thử overfit một tập rất nhỏ để phát hiện lỗi học, rồi so sánh train–validation–test. Cần ghi rõ tokenization, từ vựng, dữ liệu, seed, cấu hình mô hình và cách giải mã để người khác chạy lại.

Tài liệu tham khảo

[1] I. Sutskever, O. Vinyals, Q. V. Le, *Sequence to Sequence Learning with Neural Networks*, 2014. https://arxiv.org/abs/1409.3215.

[2] D. Bahdanau, K. Cho, Y. Bengio, *Neural Machine Translation by Jointly Learning to Align and Translate*, 2014. https://arxiv.org/abs/1409.0473.

[3] A. Vaswani et al., *Attention Is All You Need*, 2017. https://arxiv.org/abs/1706.03762.

[4] Trương Xuân Nam, *Xử lý ngôn ngữ tự nhiên*, `slide/NLP-1.pdf`, trang 6, 9–16.

[5] Notebook của dự án, `encoder_decoder_minh_hoa.ipynb`, phần 1: encoder–decoder và seq2seq.